Practical 04 --- From Notebook to Package

Turn scattered cells into a folder of code that someone else can run

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L07
Course Outcome	CO3
Lab duration	120 minutes
Memory needed	about 300 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain why a notebook stops being good enough for real work.
2. Turn cells into functions, and functions into a package of modules.
3. Import your own code and use it, the way a library is used.
4. Run your project from the command line, with no notebook at all.

What you need to know first

Everything you have written so far lives in notebook cells. That is fine for exploring. It stops being fine the moment somebody else needs to run your work.

There are three reasons.

Hidden state. A notebook remembers everything you ran, including cells you have since edited or deleted. The screen can show code that never actually ran, next to a result produced by code that no longer exists. Nobody can tell by looking.

Order. Notebook cells can be run in any order. Yours works because you happened to run them in the right sequence today. On another machine, top to bottom, it may not.

Nothing can import a notebook. A web service cannot `import` a cell. A test cannot call a cell. A scheduled job cannot run a cell. To be used by other software, code must live in a **module**: an ordinary `.py` file.

So today you refactor. **Refactoring** means changing how code is arranged without changing what it does. You will take the working code from P02, put it into functions, group those

functions into files by job, and put those files into a folder with an `__init__.py` in it. A folder like that is called a **package**, and Python can import it.

At the end you will run your whole training pipeline from a terminal, with one command and no notebook. That is the shape every later practical needs: P05 tests this package, P07 serves it, P08 puts it in a container.

Before you start

- Practicals P01-P03 are finished. You have trained and scored a model.
- You know what a function is, and what `import` does.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last
line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
```

```
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.13.5
Folder  : P04-package
  numpy          ok
  pandas         ok
  sklearn        ok
  joblib         ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so
every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the
lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
```

```
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]),
delivery_min[i]])
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ..\data\delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- See the problem before fixing it

A quick demonstration of hidden state. Watch carefully.

```
total = 10
total = total + 5
print("total is", total)
```

```
total is 15
```

Now imagine you go back, change the first line to `total = 100`, and run **only** the second line again. The screen would show `total = 100` at the top and `total is 20` at the bottom. Both are on your screen. Only one of them is true.

That is not a silly mistake. It is the normal way notebooks fail, and it is why production code does not live in them.

Step 2 --- Plan the package before writing it

Group code by **job**, not by the order you happened to write it. Our project has three jobs, so it gets three modules.

```
work/
  delivery/
    __init__.py     marks this folder as a package
    data.py         load the CSV, split it
    features.py     check and describe a single order
    model.py        train, score, save, load
  train.py          the script a human runs
```

One rule: **a module should be describable in one sentence.** If you cannot, it is doing two jobs and wants splitting.

```
from pathlib import Path

WORK = Path("work")
PKG = WORK / "delivery"
PKG.mkdir(parents=True, exist_ok=True)

print("package folder ready:", PKG.resolve())
```

```
package folder ready: C:\Users\praty\Downloads\MLOps_P04\P04-package\
work\delivery
```

Step 3 --- Write the first module: data.py

This module does one thing: get the delivery data, split into training and testing.

We write the file from the notebook using `write_text`. In real work you would type it in an editor --- we do it this way so the whole practical stays in one place.

```
data_py = '''"""Loading and splitting the delivery dataset."""

from pathlib import Path

import pandas as pd
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"
SEED = 42


def load_orders(path):
    """Read the delivery CSV into a table."""
    return pd.read_csv(Path(path))
```

```
def split_orders(orders, test_size=0.2):
    """Return X_train, X_test, y_train, y_test."""
    X = orders[FEATURES]
    y = orders[TARGET]
    return train_test_split(X, y, test_size=test_size,
                            random_state=SEED)
'''

(PKG / "data.py").write_text(data_py, encoding="utf-8")
print("wrote", PKG / "data.py", f"({len(data_py)} characters)")
```

```
wrote work\delivery\data.py (604 characters)
```

Notice what moved and what did not. The code is the same code from P02. Only its address changed.

Step 4 --- Write features.py

One job: judging and describing a single order. No file reading, no model training.

```
features_py = '''"""Checks and descriptions for one delivery order."""

TRAFFIC_LEVELS = (1, 2, 3)


def minutes_per_km(delivery_min, distance_km):
    """How many minutes each kilometre took."""
    if distance_km <= 0:
        raise ValueError("distance_km must be positive")
    return delivery_min / distance_km


def describe_order(order):
    """A short sentence a human can read."""
    weather = "in the rain" if order["rain"] else "in dry weather"
    return (f"{order['distance_km']} km, "
            f"{order['prep_time_min']} min prep, "
            f"traffic {order['traffic_level']}, {weather}")
'''

(PKG / "features.py").write_text(features_py, encoding="utf-8")
print("wrote", PKG / "features.py")
```

```
wrote work\delivery\features.py
```

Step 5 --- Write model.py

One job: everything about the model itself --- training it, scoring it, saving it to disk and loading it back.

joblib is the standard way to save a trained scikit-learn model to a file. Saving matters: training again every time you need a prediction would be absurd.

```
model_py = '''"""Training, scoring, saving and loading the model."""

import joblib
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error


def train_model(X_train, y_train):
    """Fit a linear regression and hand it back."""
    return LinearRegression().fit(X_train, y_train)


def evaluate(model, X_test, y_test):
    """Mean absolute error, in minutes."""
    return mean_absolute_error(y_test, model.predict(X_test))


def save_model(model, path):
    """Write a trained model to disk."""
    joblib.dump(model, path)
    return path


def load_model(path):
    """Read a trained model back from disk."""
    return joblib.load(path)
'''

(PKG / "model.py").write_text(model_py, encoding="utf-8")
print("wrote", PKG / "model.py")
```

```
wrote work\delivery\model.py
```

Step 6 --- Add **init**.py to make it a package

A folder of `.py` files is just a folder. Add a file called `__init__.py` and Python treats it as a **package** you can import.

It can be empty. Ours will list what the package offers, which is good manners.

```
init_py = '''"""Delivery-time prediction for SCSE3040."""

__version__ = "0.1.0"

from .data import FEATURES, TARGET, load_orders, split_orders
from .model import evaluate, load_model, save_model, train_model

__all__ = [
```

```
    "FEATURES", "TARGET", "load_orders", "split_orders",
    "train_model", "evaluate", "save_model", "load_model",
]
'''

(PKG / "__init__.py").write_text(init_py, encoding="utf-8")

for f in sorted(PKG.glob("*.py")):
    print(f"  {f.name:<15} {f.stat().st_size:>5} bytes")
```

```
  __init__.py       338 bytes
  data.py           627 bytes
  features.py       612 bytes
  model.py          696 bytes
  validate.py       363 bytes
```

Step 7 --- Import your own package and use it

Python looks for packages in a list of folders called `sys.path`. Our package is inside `work/`, so we add that folder to the list.

The `fresh_import` helper below matters: Python caches modules, so after you *edit* a file you must tell it to look again. This trips up everybody who writes modules from a notebook.

```
import importlib
import shutil
import sys

if str(WORK.resolve()) not in sys.path:
    sys.path.insert(0, str(WORK.resolve()))


def fresh_import(name):
    """Import a module, reloading it if it changed on disk.

    The first line deletes Python's compiled cache. Python decides
    whether that cache is stale from the file's size and its
    timestamp to the nearest second -- so an edit that keeps the
    length the same can be missed entirely. Deleting it is simplest.
    """
    shutil.rmtree(PKG / "__pycache__", ignore_errors=True)
    importlib.invalidate_caches()
    module = importlib.import_module(name)
    return importlib.reload(module)


delivery_data = fresh_import("delivery.data")
delivery_model = fresh_import("delivery.model")

orders = delivery_data.load_orders(DATA)
```

```
X_train, X_test, y_train, y_test = delivery_data.split_orders(orders)
model = delivery_model.train_model(X_train, y_train)
mae = delivery_model.evaluate(model, X_test, y_test)

print(f"rows {len(orders)}  train {len(X_train)}  test {len(X_test)}")
print(f"MAE {mae:.2f} minutes")
```

```
rows 600  train 480  test 120
MAE 1.92 minutes
```

Four lines of your own library did what a screen of notebook cells did in P02 --- and this time somebody else can run it.

Step 8 --- Save the trained model to a file

Training takes time. Predicting should not repeat it. Save the trained model once; load it whenever you need a prediction.

```
model_path = WORK / "model.joblib"
delivery_model.save_model(model, model_path)

reloaded = delivery_model.load_model(model_path)
same = float(reloaded.predict(X_test.head(1))[0])
original = float(model.predict(X_test.head(1))[0])

print(f"saved to {model_path} "
      f"({model_path.stat().st_size:,} bytes)")
print(f"original model predicts {original:.2f}")
print(f"reloaded model predicts {same:.2f}")
print("identical:", abs(same - original) < 1e-9)
```

```
saved to work\model.joblib (921 bytes)
original model predicts 47.65
reloaded model predicts 47.65
identical: True
```

That file is the thing P07 will serve and P08 will put in a container. From here on, "the model" means this file.

Step 9 --- Write the script a human runs

Finally, the entry point: an ordinary Python file that a person, or a scheduled job, or a container, can run with one command.

The `if __name__ == "__main__":` line means "only do this when somebody runs this file directly, not when it is imported".

```
train_py = '''"""Train the delivery-time model and report its
error."""
```

```
import sys
from pathlib import Path

from delivery import (evaluate, load_orders, save_model,
                      split_orders, train_model)


HERE = Path(__file__).resolve().parent
DEFAULT_DATA = HERE.parent.parent / "data" / "delivery_times.csv"


def main():
    data_path = Path(sys.argv[1]) if len(sys.argv) > 1 else
DEFAULT_DATA

    orders = load_orders(data_path)
    X_train, X_test, y_train, y_test = split_orders(orders)
    model = train_model(X_train, y_train)
    mae = evaluate(model, X_test, y_test)

    save_model(model, Path(__file__).parent / "model.joblib")

    print(f"rows: {len(orders)}")
    print(f"MAE: {mae:.2f}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
'''

(WORK / "train.py").write_text(train_py, encoding="utf-8")
print("wrote", WORK / "train.py")
```

```
wrote work\train.py
```

Step 10 --- Run it as a real program

No notebook involved. This is the command you would put in a Dockerfile in P08, or in a pipeline in P11.

```
import subprocess

def run_script(name, args=(), cwd=WORK):
    """Run a python file the way a terminal would, and show output."""
    done = subprocess.run(
        [sys.executable, name, *args],
        cwd=cwd, capture_output=True, text=True,
    )
    print(f"$ python {name} {' '.join(args)}".rstrip())
```

```
    print(done.stdout.strip() or "(nothing printed)")
    if done.returncode != 0:
        print("STDERR:", done.stderr.strip()[-800:])
    print("exit code:", done.returncode)
    return done

result = run_script("train.py", [str(DATA.resolve())])
```

```
$ python train.py C:\Users\praty\Downloads\MLOps_P04\data\
delivery_times.csv
rows: 600
MAE: 1.92
exit code: 0
```

Exit code `0` means success. Any other number means failure --- and that is exactly how a pipeline in P11 will decide whether to carry on or stop.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Add a function to the package

Add a function `average_speed_kmph(distance_km, delivery_min)` to `delivery/features.py`. It returns the average speed in kilometres per hour.

Speed in km/h is `distance_km / (delivery_min / 60)`.

You must **append** it to the existing file, not replace the file, then re-import the module and store the result of `average_speed_kmph(10, 30)` in `T1_speed`.

> **Hint.** Build the new function as a string and use `open(PKG / 'features.py', 'a', encoding='utf-8')` -- the `'a'` means append. Then `feat = fresh_import('delivery.features')` and call `feat.average_speed_kmph(10, 30)`. Ten km in thirty minutes is 20 km/h.

```
new_function = '''

def average_speed_kmph(distance_km, delivery_min):
```

```
    "Average speed of a delivery, in kilometres per hour."
    return distance_km / (delivery_min / 60)
'''

with open(PKG / "features.py", "a", encoding="utf-8") as fh:
    fh.write(new_function)

feat = fresh_import("delivery.features")

T1_speed = feat.average_speed_kmph(10, 30)

print("10 km in 30 minutes =", T1_speed, "km/h")
```

```
10 km in 30 minutes = 20.0 km/h
```

Task T2 --- Reject an impossible order

Write a **new module** `delivery/validate.py` containing one function, `is_valid_order(order)`, which takes a dictionary and returns `True` only if all four of these hold:

- `distance_km` is greater than 0
- `prep_time_min` is 0 or more
- `traffic_level` is 1, 2 or 3
- `rain` is 0 or 1

Anything else returns `False`. Import it and store the module in `T2_validate`.

> **Hint.** Build the module as one big string and `write_text` it, exactly the way Steps 3-5 did. Inside the function, use `order["..."]` to read each value, and `return False` as soon as any rule is broken. Then `T2_validate = fresh_import('delivery.validate')`.

```
validate_py = '''"Is this order usable?"


def is_valid_order(order):
    "True if the order passes every rule."

    if order["distance_km"] <= 0:
        return False

    if order["prep_time_min"] < 0:
        return False

    if order["traffic_level"] not in (1, 2, 3):
        return False

    if order["rain"] not in (0, 1):
        return False
```

```
    return True
'''

(PKG / "validate.py").write_text(validate_py, encoding="utf-8")

T2_validate = fresh_import("delivery.validate")

good = {"distance_km": 5.0, "prep_time_min": 20,
        "traffic_level": 2, "rain": 0}

if T2_validate is None:
    print("Not done yet -- fill in the two TODOs above.")
else:
    print("good order ->", T2_validate.is_valid_order(good))
```

```
good order -> True
```

Task T3 --- A second command-line script

Write `work/predict.py`: a script that loads the saved model from `work/model.joblib` and prints one prediction for a 7 km order with 25 minutes of preparation, traffic level 3, no rain.

It must print a line that starts with `PREDICTION:` followed by the number of minutes.

Run it with `run_script("predict.py")` and store the finished process in `T3_run`.

> **Hint.** Copy the shape of `train.py` from Step 9. Import `load_model` from `delivery`, build a one-row `pandas` DataFrame with the four columns, call `.predict(...)`, and print `f"PREDICTION: {minutes:.1f}"`. Do not forget the `if __name__ == "__main__":` guard.

```
predict_py = '''"Predict one delivery, from the command line."

from pathlib import Path
import pandas as pd

from delivery import load_model


def main():
    model_path = Path(__file__).parent / "model.joblib"
    model = load_model(model_path)

    order = pd.DataFrame([{
        "distance_km": 7,
        "prep_time_min": 25,
        "traffic_level": 3,
        "rain": 0
    }])
```

```
    minutes = model.predict(order)[0]

    print(f"PREDICTION: {minutes:.1f}")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
'''

(WORK / "predict.py").write_text(predict_py, encoding="utf-8")

T3_run = run_script("predict.py")
```

```
$ python predict.py
PREDICTION: 56.5
exit code: 0
```

Self-check

Run the cell below to mark your work.

```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | delivery.features now offers average_speed_kmph', lambda:
hasattr(feat, 'average_speed_kmph'))
_check('T1 | 10 km in 30 minutes is 20 km/h', lambda:
abs(float(T1_speed) - 20.0) < 1e-6)
_check('T1 | it still works for another order', lambda:
abs(float(feat.average_speed_kmph(6, 45)) - 8.0) < 1e-6)
_check('T2 | delivery/validate.py exists and imports', lambda: (PKG /
```

```
'validate.py').is_file() and hasattr(T2_validate, 'is_valid_order'))
_check('T2 | a sensible order is accepted', lambda:
T2_validate.is_valid_order({'distance_km': 5.0, 'prep_time_min': 20,
'traffic_level': 2, 'rain': 0}) is True)
_check('T2 | a zero distance is rejected', lambda:
T2_validate.is_valid_order({'distance_km': 0, 'prep_time_min': 20,
'traffic_level': 2, 'rain': 0}) is False)
_check('T2 | traffic level 9 and rain 5 are both rejected', lambda:
T2_validate.is_valid_order({'distance_km': 5.0, 'prep_time_min': 20,
'traffic_level': 9, 'rain': 0}) is False and
T2_validate.is_valid_order({'distance_km': 5.0, 'prep_time_min': 20,
'traffic_level': 2, 'rain': 5}) is False)
_check('T3 | predict.py ran and exited successfully', lambda:
T3_run.returncode == 0)
_check('T3 | it printed a PREDICTION line with a sensible number',
lambda: 'PREDICTION:' in T3_run.stdout and 20 <
float(T3_run.stdout.split('PREDICTION:')[1].split()[0]) < 90)

print("================================================================
===")
print("SELF-CHECK   Practical 04 --- From Notebook to Package")
print("================================================================
===")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------
---")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("================================================================
===")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell
again.")
```

```
================================================================
SELF-CHECK   Practical 04 --- From Notebook to Package
================================================================
  [PASS]  T1 | delivery.features now offers average_speed_kmph
  [PASS]  T1 | 10 km in 30 minutes is 20 km/h
  [PASS]  T1 | it still works for another order
  [PASS]  T2 | delivery/validate.py exists and imports
  [PASS]  T2 | a sensible order is accepted
  [PASS]  T2 | a zero distance is rejected
  [PASS]  T2 | traffic level 9 and rain 5 are both rejected
  [PASS]  T3 | predict.py ran and exited successfully
  [PASS]  T3 | it printed a PREDICTION line with a sensible number
----------------------------------------------------------------
```

```
  9 of 9 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. The `work/delivery/` folder you built, zipped.
3. In a markdown cell, one sentence per module saying what its single job is.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, package builds and imports	3
Task T1 --- a new function added to the package	2
Task T2 --- validation logic that rejects bad orders	3
Task T3 --- a script that runs from the command line	2
Total	**10**

Read more

- Python docs --- Modules and packages --- https://docs.python.org/3/tutorial/modules.html
- Python docs --- The module search path --- https://docs.python.org/3/tutorial/modules.html#the-module-search-path
- joblib --- Persistence --- https://joblib.readthedocs.io/en/stable/persistence.html
- scikit-learn --- Model persistence --- https://scikit-learn.org/stable/model_persistence.html